Programación III

Guía de patrones creacionales y concurrencia en C++20

Material de aprendizaje

Octubre de 2026

Sistema Concurrente y Extensible de Procesamiento de Eventos

Esta actividad propone construir, por etapas, un programa que lee mensajes de un archivo TXT, los convierte en eventos, los procesa en modo síncrono o asíncrono, entrega cada mensaje en la carpeta de su destinatario y deja constancia de lo ocurrido en un log centralizado. El modo asíncrono emplea varios workers (threads). Su propósito es comprender el uso de Singleton, Builder y Abstract Factory, y explorar Factory Method al incorporar un segundo formato. No es una actividad calificada ni requiere implementar todos los formatos desde el inicio.

1. El problema

Un sistema recibe un archivo de texto con mensajes de distinta naturaleza. Algunas líneas representan alertas y otras, información ordinaria. Cada mensaje debe convertirse en un evento con datos consistentes, procesarse, entregarse a un destinatario y registrarse. Si crece el volumen de mensajes, varios workers deben colaborar sin corromper el archivo de log ni perder eventos. El primer alcance utiliza únicamente TXT. Todos los TXT pasan por el mismo lector y la misma carpeta de entrada. Un archivo con una línea no vacía representa un mensaje individual; con dos o más líneas no vacías representa un lote. Un archivo sin líneas de mensaje no genera eventos. CSV se plantea como extensión opcional para explorar la elección de lector según el formato identificado del archivo. JSONL se menciona solo como una posibilidad futura; no forma parte de la implementación inicial.

2. Resultado que se busca observar

Al ejecutar el programa con un TXT de ejemplo, la consola debe mostrar una línea completa por evento aceptado, línea rechazada, resultado de procesamiento y entrega, seguida de un resumen final. logs/events.log conserva el mismo rastro para revisarlo después; las carpetas de destinatarios y salida/dlq/ muestran el resultado material de cada mensaje. Los registros deben incluir archivo y número de línea, identificador cuando exista, resultado y motivo de fallo. El orden de terminación puede variar entre ejecuciones con varios workers, pero los conteos finales deben coincidir.

3. Entrada inicial y reglas mínimas

Para facilitar el aprendizaje, todos los TXT usan el mismo esquema por línea: tipo|severidad|destinatario|mensaje. Por ejemplo:

```
alerta|alta|docencia|Temperatura fuera de rango
informacion|baja|soporte|Lectura recibida
```

La cantidad de líneas no vacías determina si el archivo contiene un mensaje o un lote; después, cada línea se valida de forma independiente. Los cuatro campos son obligatorios. Si falta uno, el tipo o la severidad no son válidos, el destinatario no está configurado o el mensaje está vacío, se rechaza solo esa línea, se conserva su contenido original en la DLQ y se anota el motivo con archivo y número de línea en el log. Las demás líneas del lote continúan. Si el archivo de entrada no existe o no puede abrirse, se informa en el log y en el resumen; no se genera una entrada de DLQ porque no se recibió ningún mensaje. Se recomienda asociar a cada evento aceptado un identificador y su procedencia.

4. Los patrones dentro del problema

Singleton para el log centralizado

Hay un único punto responsable de escribir el log compartido por todos los workers. La pregunta de diseño no termina al garantizar una sola instancia: también hay que proteger cada operación de escritura y definir cuándo se abre y cierra el archivo. Una inicialización segura no vuelve seguras automáticamente las operaciones posteriores.

Builder para construir objetos válidos

Cada línea aporta datos parciales que deben transformarse en un evento coherente. El Builder permite reunir campos, aplicar valores predeterminados y validar antes de entregar el objeto. El estudiante puede elegir construir eventos o la configuración de ejecución; en cualquier caso debe explicar qué estado incompleto evita.

Factory Method para elegir un lector al ampliar los formatos

En la primera etapa, todos los archivos son TXT y un solo lector basta. La oportunidad de estudiar Factory Method aparece al incorporar CSV: antes de importar cada archivo, el programa identifica un formato admitido y utiliza el lector correspondiente. Ambos lectores deben producir datos compatibles con el mismo modelo de evento, de modo que Builder, workers, destinatarios y log no dependan del formato de entrada. Identificar la extensión .txt o .csv y devolver un lector desde un condicional central es una fábrica simple. Para explorar Factory Method en sentido estricto, imagine una clase creadora que organiza el flujo común de importación y delega la creación del lector en un método que sus variantes para TXT y CSV redefinen. El método de creación pertenece al flujo compartido de importación; cada variante de la clase creadora proporciona el lector concreto que utiliza ese flujo.

Abstract Factory para familias de ejecución

El programa debe ofrecer dos modos seleccionables al iniciar. La entrada entrega eventos a un despachador, el módulo de resultados consulta su avance y el cierre espera la terminación antes de emitir el resumen. Como estas responsabilidades pertenecen a partes distintas del programa, cada modo necesita una familia compatible de despachador, seguimiento de resultados y coordinación de cierre:

- Síncrono: procesa el evento durante la llamada, registra el resultado inmediato y cierra sin tareas pendientes.
- Asíncrono: encola el evento, sigue los trabajos pendientes y cierra la admisión antes de esperar a los workers con std::jthread.

Abstract Factory crea juntos los tres componentes del modo elegido y les permite compartir el estado que necesitan. Mezclar un despachador que encola con un cierre que presupone resultados inmediatos podría imprimir el resumen antes de procesar todos los mensajes. Esta incompatibilidad hace recomendable la fábrica de familias en este proyecto; la selección del lector y del tipo de mensaje son decisiones independientes. Las decisiones se aplican en este orden:

1. Al iniciar, el modo síncrono o asíncrono determina la familia de despacho, seguimiento y cierre mediante Abstract Factory.
2. Al abrir un archivo, su formato determina el lector. Con solo TXT no hay elección; CSV añade una segunda variante.
3. Al interpretar cada registro, el campo tipo determina el tratamiento de alerta o información.
4. Al entregar el resultado, el campo destinatario se consulta en config/recipients.txt para encontrar su carpeta.

5. Concurrencia con std::jthread

Una vez que funcione el modo síncrono, incorpore el modo asíncrono con varios workers de std::jthread. En este modo, la entrega de un evento al despachador devuelve el control antes de que termine su procesamiento. El seguimiento conserva los trabajos pendientes y el cierre impide nuevas entregas, espera a que terminen los workers y solo entonces permite imprimir el resumen. Una cola compartida con mutex y condition_variable es una opción, pero se admiten otras estrategias bien explicadas. Evite la espera activa y no use pausas artificiales como mecanismo de sincronización. El log y los contadores compartidos necesitan protección. Antes de elegir un mutex o un dato atómico, identifique qué operación debe ser indivisible. El objetivo observable es que cada evento aceptado aparezca una vez en el resultado, que los registros del log sean legibles, que cada entrega o paso a DLQ quede contabilizado y que el programa termine sin quedarse esperando.

6. Entrega a destinatarios en carpetas

Cada evento indica un destinatario lógico. El programa carga al iniciar un archivo externo config/recipients.txt que relaciona nombres válidos con carpetas de salida; por ejemplo, docencia y soporte. Agregar o quitar un destinatario requiere editar ese archivo, sin modificar ni recompilar C++. Si la carpeta de un destinatario válido aún no existe, el programa debe crearla antes de la primera entrega. El TXT de mensajes solo contiene el nombre lógico; no puede indicar una ruta arbitraria. Tras procesar un evento, el sistema guarda su mensaje en la carpeta correspondiente y anota en el log central si la entrega tuvo éxito o falló. Para comenzar, pueden configurarse dos destinatarios y guardar un archivo de nombre único por evento. La carpeta configurada se crea si falta, una vez validado el destinatario. El estudiante debe decidir cómo evitar sobrescrituras cuando varios workers entregan mensajes al mismo destino.

Validación de carpetas de salida

La validación y la entrega siguen este orden:

1. Al iniciar, valide config/recipients.txt: identificadores no vacíos ni repetidos, rutas no vacías y carpetas permitidas dentro de la raíz de salida prevista. Informe una configuración inválida antes de procesar mensajes.
2. Antes de entregar, compruebe que el destinatario figura en la lista cargada. Si es desconocido, envíe el mensaje a la DLQ sin crear carpeta.
3. Para un destinatario válido, verifique su carpeta: créela si falta o reutilícela si ya existe. Puede crearla al iniciar o antes de la primera entrega, siempre antes de escribir el mensaje.
4. Si la carpeta válida no puede crearse o usarse, registre el fallo de entrega y envíe el mensaje a la DLQ.

Carpeta DLQ para mensajes fallidos

Una línea de mensaje inválida o un mensaje válido que no pueda entregarse debe apartarse en salida/dlq/ como archivo individual de nombre único, por ejemplo failed_0003.txt. Un archivo de entrada inexistente se reporta, pero no produce archivo en la DLQ. El registro de DLQ debe conservar el contenido original, archivo y línea de entrada, y motivo del fallo, de modo que pueda revisarse o reintentarse más adelante. El log central registra también el fallo y la ubicación del archivo en DLQ. Si la propia DLQ no puede escribirse, el programa debe informar el problema y contabilizarlo, sin ocultar la pérdida.

7. Ruta sugerida de trabajo

1. Defina en pocas líneas el formato TXT, los datos del evento y cómo se informarán los errores. Prepare un archivo pequeño con líneas válidas e inválidas.
2. Lea el TXT de forma secuencial y muestre qué evento se obtendría de cada línea. Compruebe que un error en una línea no impide continuar con las siguientes.
3. Incorpore el Builder y pruebe qué ocurre cuando faltan datos o se proporciona un valor no admitido.
4. Separe el tratamiento de alerta y el de información. Use una selección central sencilla por el campo tipo.
5. Centralice el registro mediante Singleton y compruebe primero las anotaciones de una ejecución síncrona.
6. Añada std::jthread y una forma de distribuir eventos. Compruebe la terminación, los conteos y la integridad del log con varias ejecuciones.
7. Cree config/recipients.txt con dos destinatarios y carpetas diferentes; añada un tercero modificando solo la configuración y compruebe que su carpeta se crea automáticamente. Incorpore la carpeta DLQ. Compruebe entregas correctas y fallidas con un TXT de una línea y otro de varias; cada error debe conservar el mensaje original y su causa.
8. Mantenga disponibles ambos modos. Use Abstract Factory para crear, al iniciar, la familia compatible de despacho, seguimiento y cierre. Compruebe que el modo asíncrono no emite el resumen antes de vaciar sus trabajos pendientes.
9. Si desea ampliar la entrada, incorpore CSV. Identifique primero el formato del archivo, seleccione el lector apropiado y produzca el mismo modelo de evento que genera el lector TXT.

8. Experimentos para comprobar lo aprendido

- Use un TXT de un solo mensaje y otro con varias líneas. Luego pruebe un lote con líneas vacías, tipos distintos, un mensaje vacío y un tipo desconocido. Revise qué aceptó el programa y qué anotó como error.
- Ejecute el mismo archivo con un worker y luego con varios. Compare cantidades e integridad del log; no exija el mismo orden de resultados.
- Repita varias veces la ejecución con un archivo grande. Busque eventos duplicados, ausentes, líneas mezcladas o cierres incompletos.
- Añada y elimine un destinatario en config/recipients.txt sin cambiar el código. Pruebe que la carpeta del nuevo destinatario se crea si falta. Compruebe con dos destinatarios que los archivos se guardan en carpetas distintas; pruebe un destinatario desconocido, un fallo de entrega y un nombre de archivo repetido. Revise la DLQ y compruebe que el log identifica el resultado y la causa de cada fallo.
- Añada un tercer tipo de evento o ejecute los mismos mensajes en los modos síncrono y asíncrono. Compruebe que los resultados y conteos finales coinciden.

9. Material de apoyo recomendado

Conserve el código, un README breve con instrucciones de compilación y ejecución, un TXT de prueba, config/recipients.txt, carpetas de destinatarios y un esquema sencillo de clases o componentes. Un párrafo por patrón que explique la decisión tomada y sus alternativas será más útil que una lista de nombres de clases. CMake y pruebas automatizadas son recomendables para repetir los experimentos, pero la actividad no tiene rúbrica ni puntuación.

10. Ampliaciones opcionales

CSV puede incorporarse como segundo formato de entrada cuando el flujo TXT ya esté claro. Cada archivo se identifica por un formato admitido y se dirige a su lector; ambos lectores producen eventos del mismo modelo. JSONL y otros escenarios son caminos posteriores. Ninguna ampliación es necesaria para completar el recorrido inicial.

Apéndice A Archivos de mensajes para practicar

Los ejemplos siguientes son contenido de archivos TXT. El mismo lector recibe todos los archivos: la cantidad de líneas no vacías determina si hay un mensaje o un lote. Los nombres son arbitrarios y no activan modos distintos. Las carpetas asociadas a docencia y soporte se definen en la configuración, no en el mensaje.

messages_01.txt

Contiene un solo mensaje dirigido a docencia:

```
alerta|alta|docencia|Temperatura fuera de rango
```

Se espera un evento aceptado, una entrega en la carpeta de docencia y un registro de resultado en el log central.

messages_02.txt

Contiene cuatro mensajes independientes, destinados a carpetas distintas:

```
alerta|alta|docencia|Temperatura fuera de rango
informacion|baja|soporte|Lectura recibida
alerta|media|soporte|Servicio sin respuesta
informacion|baja|docencia|Reporte disponible
```

Se esperan cuatro eventos aceptados. El orden de entrega puede variar si se ejecutan varios workers, pero cada mensaje debe entregarse una sola vez en la carpeta de su destinatario y quedar contabilizado en el resumen.

messages_03.txt

Permite observar cómo se continúa después de un registro inválido:

```
informacion|baja|soporte|Inicio del lote
alerta|alta||Falta el destinatario
alerta|alta|desconocido|Destinatario no configurado
informacion|baja|docencia|Fin del lote
```

El estudiante debe distinguir entre un mensaje mal formado y un destinatario no configurado. Ambos registros fallidos deben guardarse por separado en salida/dlq/ con su motivo y procedencia, sin impedir que se atiendan los mensajes válidos del lote. La clasificación exacta de esos errores debe documentarse.

Apéndice B Estructura sugerida de carpetas

Esta organización separa los archivos que llegan, la lista editable de destinatarios, las entregas y el log. Las carpetas docencia y soporte son ejemplos del archivo de configuración: pueden añadirse o eliminarse sin tocar el código.

```
proyecto/
  config/
    recipients.txt
  entrada/
    messages_01.txt
    messages_02.txt
    messages_03.txt
  salida/
    destinatarios/
      docencia/
        event_0001.txt
      soporte/
        event_0002.txt
    dlq/
      failed_0003.txt
  logs/
    events.log
```

Los tres TXT de entrada comparten carpeta y lector. El primero tiene una línea no vacía; los otros tienen varias. Cada línea válida puede producir un archivo de entrega con nombre único en la carpeta asignada al destinatario. El ejemplo muestra docencia y soporte, pero el programa obtiene la relación entre nombre lógico y carpeta desde config/recipients.txt. El log central permanece en logs/events.log: no se crea un log diferente por destinatario. Un destinatario desconocido o una entrega fallida se registra como error y se conserva en salida/dlq/; el texto recibido no debe crear carpetas arbitrarias. Si dos workers intentan entregar mensajes al mismo destinatario, el diseño debe evitar que uno sobrescriba el archivo del otro.

Apéndice C Valores válidos y casos de validación

Para comparar las pruebas, esta guía fija valores concretos para los tres primeros campos de tipo|severidad|destinatario|mensaje. Cada campo se valida de forma independiente antes de intentar la entrega.

Tipo de mensaje

- alerta: mensaje que requiere el tratamiento de una alerta.
- informacion: mensaje informativo ordinario.

Cualquier otro valor en el primer campo es inválido y debe enviarse a la DLQ con la línea original y el motivo. El tipo determina qué tratamiento se selecciona; esa selección por sí sola no exige Factory Method.

Severidad del mensaje

- baja, media, alta: valores permitidos en el segundo campo.

La severidad caracteriza el mensaje, independientemente de su tipo. Un valor ausente o distinto de esos tres se rechaza y se conserva en la DLQ.

Configuración externa de destinatarios

El archivo config/recipients.txt puede usar una relación por línea con el esquema destinatario| carpeta. Un ejemplo inicial es:

```
docencia|salida/destinatarios/docencia/
```

```
soporte|salida/destinatarios/soporte/
```

Estas líneas son datos de configuración, no nombres codificados en C++. Un destinatario es válido solo si aparece en la lista cargada al inicio de la ejecución. Para probar la extensibilidad, añada por ejemplo biblioteca|salida/destinatarios/biblioteca/ sin crear antes la carpeta; compruebe que el programa la crea y puede recibir mensajes sin recompilar. Un nombre ausente de la lista pasa a la DLQ; no se crean carpetas arbitrarias a partir del TXT de entrada.

Registro central

Para este ejercicio no se exige un campo adicional de tipo de log ni una escala separada de niveles de log. Cada anotación en logs/events.log debe permitir reconocer cuándo ocurrió algo, qué mensaje o línea de entrada afectó, a qué destinatario iba, si se entregó o falló y cuál fue la causa del error. Puede añadirse una clasificación más detallada en una extensión futura si hace falta filtrar el registro por etapas.

Casos que conviene comprobar

- Tipo, severidad y destinatario válidos, con carpeta ya existente: el mensaje se entrega una vez y el log registra el éxito.
- Destinatario válido con carpeta aún inexistente: se crea la carpeta configurada, se entrega el mensaje y se registra el éxito.
- Campo ausente, mensaje vacío, tipo o severidad inválidos, o destinatario no configurado: solo esa línea se guarda en la DLQ y el log explica el motivo; las demás líneas del archivo continúan.
- Archivo de entrada inexistente o ilegible: se informa el error en el log y el resumen, sin crear archivo de DLQ.
- Configuración con identificador duplicado o ruta inválida: se informa el error al iniciar y no se procesan mensajes.
- La carpeta de un destinatario válido no puede crearse o no permite escribir: no se declara éxito; se intenta guardar el mensaje en la DLQ y el log informa el fallo de entrega.
- Escritura fallida también en la DLQ: el programa informa el problema y lo refleja en el resumen, sin ocultar la pérdida.

Apéndice D Cómo observar una ejecución

Use messages_03.txt del Apéndice A con docencia y soporte configurados. La consola puede usar etiquetas como las siguientes; son un ejemplo de salida observable, no una interfaz de código obligatoria.

```
ACCEPTED  | messages_03.txt:1 | event_0001 | informacion | soporte
REJECTED  | messages_03.txt:2 | missing recipient | dlq/failed_0001.txt
REJECTED  | messages_03.txt:3 | unknown recipient | dlq/failed_0002.txt
ACCEPTED  | messages_03.txt:4 | event_0002 | informacion | docencia
PROCESSED | event_0001 | result=ok
PROCESSED | event_0002 | result=ok
DELIVERED | event_0001 | salida/destinatarios/soporte/event_0001.txt
DELIVERED | event_0002 | salida/destinatarios/docencia/event_0002.txt
SUMMARY   | lines=4 accepted=2 rejected=2 processed=2 delivered=2 dlq=2
```

Las líneas ACCEPTED indican que el registro superó la validación; REJECTED identifica el archivo, la línea, el motivo y el archivo de DLQ. PROCESSED muestra que terminó el tratamiento del evento; DELIVERED señala dónde quedó el mensaje. SUMMARY reúne los conteos y debe imprimirse cuando todos los workers hayan terminado. En una ejecución concurrente, el orden de PROCESSED y DELIVERED puede variar, pero ningún evento debe faltar ni repetirse. Después de la ejecución, compare la consola con logs/events.log y compruebe que existen dos archivos de entrega y dos archivos en salida/dlq/. El log debe conservar el resultado con su procedencia; el contenido de cada archivo de entrega o DLQ permite verificar el mensaje concreto. Si una carpeta válida falta al inicio, la ejecución también permite observar su creación antes de la entrega.

Apéndice E Análisis final de las decisiones de diseño

- ¿Qué dato pertenece al evento y cuál pertenece a la configuración general del programa?
- ¿Qué debe ocurrir si una línea es inválida: detener todo el proceso o registrar el error y continuar?
- ¿Dónde se encuentra la decisión sobre el tipo de tratamiento? ¿Cómo se añade uno nuevo sin confundir esa selección con la elección del lector?
- ¿Qué estado comparten el despacho, el seguimiento y el cierre de cada modo? ¿Qué fallo produciría mezclar componentes de familias distintas?
- ¿Por qué tener una sola instancia del log no garantiza que dos escrituras simultáneas sean seguras?
- ¿Qué condición indica a los workers que pueden terminar? ¿Cómo se evita perder un evento aún pendiente?
- ¿Cómo se carga la lista externa de destinatarios válidos y su relación con las carpetas? ¿En qué momento se crea una carpeta válida que aún no existe? ¿Qué sucede si su creación o la entrega falla?
- ¿Qué valores se aceptan en los campos tipo y severidad, y cómo se informa un valor desconocido?
- Al admitir CSV, ¿qué pasos de importación permanecieron iguales, cuál fue la interfaz común de los lectores y qué decisión cambió? Si la jerarquía solo repite una línea de creación, ¿sería más clara una fábrica simple?
- ¿Qué cambió al añadir un nuevo tipo, formato o contexto? ¿Qué patrón redujo cambios reales y cuál añadió estructura sin beneficio observable?